

HTTP Methods & Status Codes

Video 2 of "REST API Mastery" series — Easy English study notes to help you understand and explain REST APIs confidently in interviews.

Backend

MERN

Interview Prep

The 5 Core HTTP Methods

REST APIs use HTTP methods to tell the server **what action** to perform on a resource. There are 5 methods you must know.

Easy Analogy: Think of a notebook. GET is like reading a page. POST is like adding a new page. PUT/PATCH is like editing a page. DELETE is like tearing a page out.

Method	Action
GET	Fetch / read data
POST	Create a new resource
PUT	Update or fully replace a resource
PATCH	Partially update a resource
DELETE	Remove a resource

Simple line to remember: These 5 methods map directly to CRUD — Create, Read, Update, Delete.

How to answer in interview: "The core HTTP methods are GET, POST, PUT, PATCH, and DELETE. Together they map to CRUD operations on a resource."

PUT vs POST

This is one of the most asked questions. The difference is about **idempotency** — does repeating the same request change the result again and again, or not?

Easy Analogy: POST is like placing a new order at a restaurant — order it twice, you get two meals. PUT is like updating your seat number — do it 5 times with the same value, you still have just one seat number.

PUT VS POST

POST /users

Creates a NEW user every time

PUT /users/123

Updates SAME user every time

Simple line to remember: PUT is idempotent (same result every time). POST is NOT idempotent (creates something new each time).

How to answer in interview: "PUT is idempotent — calling it multiple times gives the same result, so it's used to update an existing resource. POST is not idempotent — every call creates a new resource."

HTTP Status Codes

Status codes tell the client what happened with their request. They are grouped into 3 main ranges.

Range	Meaning	Common Examples
2xx	Success	200 OK, 201 Created, 204 No Content
4xx	Client made a mistake	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server made a mistake	500 Internal Server Error, 503 Service Unavailable

Easy Analogy: Think of status codes like traffic signals. 2xx is green (go, success). 4xx is a mistake you made while driving (wrong turn). 5xx is the road itself being broken (not your fault).

How to answer in interview: "2xx codes mean success, 4xx codes mean the client sent a bad request, and 5xx codes mean something failed on the server side."

Safe vs Idempotent Methods

These two words sound similar but mean different things.

- 1 **Safe methods** never change anything on the server. Example: GET, HEAD.
- 2 **Idempotent methods** can change something, but repeating them gives the same result. Example: PUT, DELETE.
- 3 **POST** is neither safe nor idempotent — it always creates something new.

QUICK MAP

GET Safe + Idempotent	PUT Idempotent only	DELETE Idempotent only	POST Neither
---------------------------------	-------------------------------	----------------------------------	------------------------

How to answer in interview: "GET is both safe and idempotent since it doesn't change data. PUT and DELETE are idempotent but not safe. POST is neither, because it creates a new resource every time."

Choosing the Right Status Code

A common mistake freshers make is returning the wrong status code. Follow this simple guide:

- 1 Successful GET → **200 OK**
- 2 Resource created (POST success) → **201 Created**
- 3 Update/Delete success with no data to return → **204 No Content**
- 4 Invalid input from client → **400 Bad Request**
- 5 Missing/invalid login → **401 Unauthorized**
- 6 Resource not found → **404 Not Found**
- 7 Unexpected server crash → **500 Internal Server Error**

Example: If a user signs up with an invalid email, return **400 Bad Request** — not 500 — because the mistake is from the client side.

How to answer in interview: "I choose status codes based on who caused the issue — 2xx for success, 4xx when the client made a mistake, and 5xx when the server fails unexpectedly."

CODEWITHHARDEEP

Liked these notes? **There's more coming.**

This was Video 2 of the "REST API Mastery" series. Next up: Core REST principles like statelessness, CORS, and the 6 architectural constraints — all explained with easy examples.

- ✓ Subscribe to CodeWithHardeep on YouTube
- ✓ Turn on notifications so you don't miss the next video
- ✓ Comment your doubts — I reply to every comment
- ✓ Watch Video 1 (REST Basics) if you haven't already

Subscribe to CodeWithHardeep ▶